Popover / Dropdown Positioning (flip on edge)

Often asked at: Atlassian, Microsoft
Tech stack: React (Vite, JavaScript, Tailwind CSS)
Time limit: 60 minutes
Core idea: `getBoundingClientRect` + viewport math: place, flip, shift, size.

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Split them into functional and non-functional. Ask clarifying questions, then write down your assumptions.
2. **List the interactions / use cases.** Every user action and what should happen. This becomes your checklist.
3. **Design the architecture.** Main components and their single responsibility.
4. **Design the state.** What is stored, what is derived, who owns it.
5. **Design the data contracts.** Inputs and outputs of each module.
6. **Trace one data flow end to end.** Input → state → side effect → output.
7. **Find the hard parts.** Solve them on paper before coding.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.**
10. **Wrap up.** Trade-offs and what you would do with more time.

Time budget for 60 minutes

Phase	Time
Requirements, clarifying questions, assumptions	5 min
Use cases + architecture + state	7 min
Pure positioning function (place, flip, shift)	15 min
Hook: measure + listen to scroll / resize	10 min
Popover component: portal, open/close, outside click, Escape	13 min
Demo page (corners, scroll box), test edge cases	7 min
Wrap-up: trade-offs, what's next	3 min

Think out loud

Say every decision before you code it: "*I'll keep the math as a pure function so I can test it without a browser.*" The interviewer grades your reasoning as much as your code. If you get stuck, say what you are trying and

why. Silence looks like being lost.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- A trigger button opens a popover (dropdown / tooltip-like panel) next to it.
- Preferred **placement**: `top`, `bottom`, `left`, `right`; **alignment** on the cross axis: `start`, `center`, `end`.
- **Flip:** if there is not enough room on the preferred side, open on the opposite side.
- **Shift:** never go off-screen on the cross axis; slide inside the viewport with a small padding.
- Stay attached to the trigger while the page or any scroll container scrolls, and on window resize.
- Close on outside click, on Escape, and on clicking the trigger again.

Non-functional requirements

Area	Requirement
Performance	At most one measure per animation frame on scroll/resize; no layout thrashing.
No flicker	Position computed before the browser paints; never shown at (0, 0) first.
Error handling	Popover bigger than the viewport → cap the height and scroll the content.
Accessibility	`aria-expanded`, `aria-controls`, `role="dialog"`, focus moves in on open and back to the trigger on Escape.
Reusability	One `<Popover>` component; the math is a pure function usable by tooltips, menus, selects.
Extensibility	Adding an arrow, auto-placement or virtual anchors later must not need a rewrite.

Clarifying questions to ask

1. Is it click-to-open (dropdown) or hover (tooltip)? Both use the same positioning.
2. Which placements must be supported? Only top/bottom, or all four sides + alignment?
3. Should it stay attached while scrolling, or close on scroll?
4. Can the trigger sit inside `overflow: hidden` / scroll containers? (Decides portal vs inline.)
5. What if the popover is taller than the space on both sides?
6. Can I use a library like Floating UI / Popper? (Usually no: they want to see the math.)

Assumptions we build with

- Click to open, one popover open at a time.
- All four sides + three alignments, default `bottom` / `center`, 8px gap (`offset`) and 8px viewport `padding`.

- Stays attached while scrolling; hides when the trigger scrolls out of the viewport.
- Rendered in a **portal** on `document.body` with `position: fixed`, so ancestors' `overflow: hidden` cannot clip it.
- No positioning library; plain React + `getBoundingClientRect`.

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Click trigger (closed)	Popover opens on the preferred side, focus moves into it.
Click trigger (open)	Popover closes.
Trigger near bottom edge, placement `bottom`	Flips to `top`.
Trigger near right edge, placement `right`	Flips to `left`.
Trigger near left edge, align `center`	Slides right so it stays 8px inside the viewport (shift).
Not enough room above or below	Stays on the bigger side, `maxHeight` capped, content scrolls.
Scroll the page or a scroll container	Popover follows the trigger; re-flips if needed.
Trigger scrolled out of view	Popover hidden.
Resize window	Re-measured and re-positioned.
Popover content changes size	Re-positioned (ResizeObserver).
Click outside	Closes, focus stays where the user clicked.
Press Escape	Closes, focus returns to the trigger.
Change placement / align in the demo	Open popover re-positions with the new options.

---

3. Design the architecture

Module	Responsibility
`computePosition` (util)	**Pure** math: trigger rect + popover size + viewport → `{ top, left, placement, maxHeight, hidden }`.
`usePopoverPosition` (hook)	Measures the DOM, calls `computePosition`, re-runs on scroll / resize / size change.
`Popover` (component)	Trigger button + open state + ARIA; renders the panel in a portal when open.
`PopoverPanel` (component)	The floating box: uses the hook, handles outside click / Escape / focus.
`App` (demo)	Triggers in all four corners, center controls, a scroll box.

```
App
├── Popover (corner × 4, center, scroll box)
│   ├── <button> trigger              ← owns `open`
│   └── createPortal → document.body
│       └── PopoverPanel              ← mounted only while open
│           └── usePopoverPosition    ← measure + listen
│               └── computePosition   ← pure math (unit tested)
```

Start simple: first `Popover` with a hard-coded `top/left`, then pull the math out into `computePosition` once flip and shift appear. The DOM measuring goes in a hook so the component stays readable.

---

4. Design the state / data model

State or field	Purpose
`open` (in `Popover`)	Is the panel mounted.
`position` (in hook)	Last computed `{ top, left, placement, maxHeight, hidden }`; `null` until first measure.
`triggerRef`, `panelRef`	DOM nodes to measure. Refs, not state: changing them must not re-render.
`placement`, `align`, `offset`, `padding`	Props (configuration), not state.

```
// Popover
const [open, setOpen] = useState(false);
const triggerRef = useRef(null);
const id = useId(); // links aria-controls ↔ panel id

// usePopoverPosition
const [position, setPosition] = useState(null); // null = not measured yet → hidden
```

Senior note

- **Derived, not stored:** the actual side (`placement` after flip), `maxHeight` and `hidden` are all computed from rects every measure. Never store "isFlipped" separately: it would go stale on the next scroll.
- **Ownership:** `Popover` owns `open` (it controls the trigger). The panel owns `position` because position only exists while the panel is mounted. Mounting the panel only when open means position resets to `null` on every open, so no stale position from last time.
- Rects are read fresh from the DOM each time; they are not cached in state.

---

5. Design the data structures and contracts

Inputs: rects from the DOM

```
// trigger.getBoundingClientRect() → viewport coordinates (already scroll-adjusted)
const triggerRect = { top: 700, left: 400, width: 100, height: 30, bottom: 730, right: 500 };
const popoverSize = { width: 256, height: 180 };          // natural size of the panel
const viewport    = { width: 1280, height: 800 };         // documentElement.clientWidth/Height
```

Why viewport coordinates: `getBoundingClientRect` already gives them, and `position: fixed` uses the same coordinate system. So **no `scrollX/scrollY` math is needed**.

Contract: `computePosition`

```
computePosition({
  triggerRect,          // required, DOMRect-like
  popoverSize,          // required, { width, height }
  viewport,             // required, { width, height }
  placement = 'bottom', // 'top' | 'bottom' | 'left' | 'right'
  align = 'center',     // 'start' | 'center' | 'end'
  offset = 8,           // gap between trigger and popover
  padding = 8,          // minimum gap to the viewport edge
})
// → { top: 512, left: 322, placement: 'top', maxHeight: 684, hidden: false }
```

Contract: `<Popover>`

```
<Popover label="Open" placement="right" align="start" offset={8} padding={8}>
  {/* any content */}
</Popover>
```

There is no API in this problem; the "mock data" is the menu list in the demo.

---

6. Trace the data flow

Key interaction: **user clicks a trigger that sits near the bottom of the screen.**

```
click trigger
   ↓
setOpen(true)                              (state change)
   ↓
render: createPortal(<PopoverPanel/>)      panel in DOM, visibility: hidden, top/left 0
```

```
   ↓
useLayoutEffect (before paint)              (side effect: measure)
   triggerRect = trigger.getBoundingClientRect()   → bottom: 730
   popoverSize = { 256, scrollHeight }              → height: 180
   viewport    = { 1280, 800 }
   ↓
computePosition (pure)
   space below = 800 - 730 = 70  <  180 + 8 + 8  → doesn't fit
   space above = 700  > 70                       → FLIP to 'top'
   top  = 700 - 8 - 180 = 512
   left = clamp(center, 8, 1280 - 256 - 8)        → SHIFT
   ↓
setPosition({...})  → re-render synchronously, still before paint
   ↓
browser paints the popover already above the trigger (no flicker)
   ↓
useEffect: focus panel; listen for outside click / Escape
scroll / resize / ResizeObserver → rAF → measure again → same pipeline
```

---

7. Important technical challenges

Challenge 1: Flip and shift without bugs

Problem. Trigger at the bottom-left corner, `placement="bottom"`, `align="center"`. The popover goes below the screen *and* half off the left edge. A naive "if it overflows, flip" can also flip to a side that is even smaller, or flip back and forth while scrolling.

Solution. Treat the two axes separately.

- **Main axis (flip):** flip only when the preferred side is too small **and** the opposite side has more room. If neither fits, stay on the bigger side.
- **Cross axis (shift):** compute the aligned position, then `clamp` it into `[padding, viewport - size - padding]`.

```
const needed = (isVertical(placement) ? p.height : p.width) + offset + padding;
const side =
  spaceOn(placement, t, vp) < needed && spaceOn(opposite, t, vp) > spaceOn(placement, t, vp)
    ? opposite
    : placement;

let { top, left } = coordsFor(side, align, t, p, offset);
if (isVertical(side)) left = clamp(left, padding, vp.width - p.width - padding);
else top = clamp(top, padding, vp.height - p.height - padding);
```

Challenge 2: Measuring without flicker

Problem. You need the popover's size to position it, but it has no size until it is rendered. With `useEffect`, the user sees one frame of the popover at (0, 0), then it jumps.

Solution. Render the panel with `visibility: hidden` (it still has layout size), measure in `useLayoutEffect` (runs after DOM update, before paint), `setPosition`, and React re-renders synchronously before the browser paints.

```
style={{ position: 'fixed', top: position?.top ?? 0, left: position?.left ?? 0,
         visibility: !position || position.hidden ? 'hidden' : 'visible' }}

useLayoutEffect(() => {
  update(); // measure + setPosition before paint
}, [update]);
```

Challenge 3: Staying attached on scroll / resize (cheaply)

Problem. The trigger is inside a scroll box. The user scrolls the box (not the window): a plain `window.onscroll` never fires, and the popover floats away. Measuring on every scroll event (can be 100+/sec) forces layout each time.

Solution.

- Listen to `scroll` on `window` with **capture = true**: scroll events don't bubble, but capture catches the scroll of *any* ancestor.
- Throttle with `requestAnimationFrame`: many events → one measure per frame.
- `ResizeObserver` on trigger and panel: content that loads late or changes size also re-positions.

```
const schedule = () => {
  cancelAnimationFrame(frame);
  frame = requestAnimationFrame(update);
};
window.addEventListener('scroll', schedule, true);
window.addEventListener('resize', schedule);
const observer = new ResizeObserver(schedule);
observer.observe(triggerRef.current);
observer.observe(popoverRef.current);
```

Challenge 4: Escaping `overflow: hidden` and capping height without feedback loops

Problem 1. The trigger is in a card with `overflow: hidden`. An absolutely positioned popover inside it gets clipped.
Solution. `createPortal(panel, document.body)` + `position: fixed` with viewport coordinates.

Problem 2. We set `maxHeight` when it doesn't fit. Next measure reads the *capped* height, now it "fits", we remove the cap, it overflows again… an oscillation.

Solution. Measure the *natural* height: `scrollHeight` is the full content height even when `maxHeight` clips it.

```
const popoverSize = {
  width: popover.offsetWidth,
  height: popover.scrollHeight + popover.offsetHeight - popover.clientHeight, // + borders
};
```

---

8. Implement in the correct order

#	Step	Working result
1	`Popover` with `open` state, trigger button, panel rendered inline	Toggle works.
2	Move panel into `createPortal(…, document.body)` with `position: fixed`	Not clipped by parents.
3	Measure trigger with `getBoundingClientRect`, place panel below it	Basic bottom placement.
4	Extract `computePosition` (pure) with 4 sides + 3 alignments	All placements work.
5	Add flip on main axis	Corners flip correctly.
6	Add shift (clamp) on cross axis	Never off-screen sideways.
7	Move measuring to `usePopoverPosition` with `useLayoutEffect` + hidden-until-measured	No flicker.
8	Scroll (capture) + resize + ResizeObserver, rAF throttled	Follows the trigger.
9	`maxHeight` when neither side fits; measure natural height	Huge content scrolls.
10	Outside click, Escape, focus in/out, ARIA attributes	Accessible and closable.
11	Hide when trigger is out of view	No floating orphan.
12	Demo page + unit tests of `computePosition`	Verified.

---

9. Cover all states and edge cases

State / edge case	Handling
Initial (closed)	Only the trigger; `aria-expanded="false"`.
Opening (not measured yet)	Panel mounted with `visibility: hidden`, `position = null`.
Open, fits on preferred side	Placed on preferred side.
Preferred side too small	Flip to the opposite side.
Both sides too small	Bigger side, `maxHeight` capped, content scrolls.
Too tall even for `top`	Pinned to `padding`, `maxHeight` = space above.

State / edge case	Handling
Close to a side edge	Shift (clamp) inside the viewport padding.
Popover wider than viewport	Clamp pins it to the left padding (documented limit).
Trigger scrolled out of view	`hidden: true` → `visibility: hidden`.
Trigger inside a scroll container	Capture-phase scroll listener catches it.
Window resize / rotate	`resize` listener re-measures.
Content size changes while open	`ResizeObserver` re-measures.
Page scrollbar present	Viewport uses `documentElement.clientWidth`, which excludes it.
Click outside / on another trigger	Closes (`pointerdown` outside both nodes).
Click trigger while open	Toggles closed; the outside handler ignores the trigger.
Escape	Closes and returns focus to the trigger.
Unmount while open	Effects clean up listeners, rAF and observer.
Placement prop changes while open	`update` changes → layout effect re-runs.

10. Final structure

```
_85_popover_positioning/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── index.css                  # @import "tailwindcss";
    ├── App.jsx                    # demo: corners, controls, scroll box
    ├── components/
    │   └── Popover.jsx            # Popover + PopoverPanel (portal)
    ├── hooks/
    │   └── usePopoverPosition.js  # measure + listeners
    └── utils/
        ├── computePosition.js     # pure place / flip / shift / size
        └── computePosition.test.js # node:assert tests
```

11. Full working code

`src/utils/computePosition.js`

```
const OPPOSITE = { top: 'bottom', bottom: 'top', left: 'right', right: 'left' };

const isVertical = (side) => side === 'top' || side === 'bottom';
const clamp = (value, min, max) => Math.max(min, Math.min(value, max));

// Free space between the trigger and the viewport edge on one side.
function spaceOn(side, t, vp) {
  return { top: t.top, bottom: vp.height - t.bottom, left: t.left, right: vp.width - t.right }[side];
}

// Main axis = which side of the trigger. Cross axis = start / center / end alignment.
function coordsFor(side, align, t, p, offset) {
  if (isVertical(side)) {
    const top = side === 'bottom' ? t.bottom + offset : t.top - offset - p.height;
    const left = align === 'start' ? t.left : align === 'end' ? t.right - p.width : t.left + (t.width -
p.width) / 2;
    return { top, left };
  }
  const left = side === 'right' ? t.right + offset : t.left - offset - p.width;
  const top = align === 'start' ? t.top : align === 'end' ? t.bottom - p.height : t.top + (t.height -
p.height) / 2;
  return { top, left };
}

/**
 * Pure: rects in, coordinates out. No DOM, so it is unit-testable.
 * triggerRect: getBoundingClientRect() of the trigger (viewport coords)
 * popoverSize: { width, height } natural size of the popover
 * viewport:    { width, height }
 */
export function computePosition({
  triggerRect: t,
  popoverSize: p,
  viewport: vp,
  placement = 'bottom',
  align = 'center',
  offset = 8,
  padding = 8,
}) {
  // 1. Flip: only when the preferred side is too small AND the opposite side has more room.
  const needed = (isVertical(placement) ? p.height : p.width) + offset + padding;
  const opposite = OPPOSITE[placement];
```

```
  const side =
    spaceOn(placement, t, vp) < needed && spaceOn(opposite, t, vp) > spaceOn(placement, t, vp)
      ? opposite
      : placement;

  let { top, left } = coordsFor(side, align, t, p, offset);

  // 2. Shift: slide along the cross axis so the popover stays inside the viewport.
  if (isVertical(side)) left = clamp(left, padding, vp.width - p.width - padding);
  else top = clamp(top, padding, vp.height - p.height - padding);

  // 3. Size: if neither side fits, cap the height and let the content scroll.
  const maxHeight = Math.max(
    0,
    isVertical(side) ? spaceOn(side, t, vp) - offset - padding : vp.height - 2 * padding
  );
  if (side === 'top') top = Math.max(top, padding);

  // Trigger scrolled out of the viewport → hide instead of floating over unrelated content.
  const hidden = t.bottom < 0 || t.top > vp.height || t.right < 0 || t.left > vp.width;

  return { top, left, placement: side, maxHeight, hidden };
}
```

src/hooks/usePopoverPosition.js

```
import { useCallback, useLayoutEffect, useState } from 'react';
import { computePosition } from '../utils/computePosition';

export function usePopoverPosition(triggerRef, popoverRef, { placement, align, offset, padding }) {
  const [position, setPosition] = useState(null);

  const update = useCallback(() => {
    const trigger = triggerRef.current;
    const popover = popoverRef.current;
    if (!trigger || !popover) return;
    // scrollHeight ignores our own maxHeight, so capping the height never feeds back into the next
measure
    const popoverSize = {
      width: popover.offsetWidth,
      height: popover.scrollHeight + popover.offsetHeight - popover.clientHeight,
    };
    // clientWidth/Height exclude the page scrollbar
    const viewport = {
      width: document.documentElement.clientWidth,
      height: document.documentElement.clientHeight,
```

```
    };
    setPosition(
      computePosition({
        triggerRect: trigger.getBoundingClientRect(),
        popoverSize,
        viewport,
        placement,
        align,
        offset,
        padding,
      })
    );
  }, [triggerRef, popoverRef, placement, align, offset, padding]);

  useLayoutEffect(() => {
    update(); // layout effect = measured before paint, so no flash at (0, 0)

    let frame = 0;
    const schedule = () => {
      cancelAnimationFrame(frame);
      frame = requestAnimationFrame(update); // at most one measure per frame
    };
    window.addEventListener('scroll', schedule, true); // capture: catches scroll of ANY ancestor
    window.addEventListener('resize', schedule);
    const observer = new ResizeObserver(schedule); // content or trigger changed size
    observer.observe(triggerRef.current);
    observer.observe(popoverRef.current);

    return () => {
      cancelAnimationFrame(frame);
      window.removeEventListener('scroll', schedule, true);
      window.removeEventListener('resize', schedule);
      observer.disconnect();
    };
  }, [update, triggerRef, popoverRef]);

  return position;
}
```

src/components/Popover.jsx

```
import { useCallback, useEffect, useId, useRef, useState } from 'react';
import { createPortal } from 'react-dom';
import { usePopoverPosition } from '../hooks/usePopoverPosition';

// Mounted only while open, so position state starts fresh on every open.
```

```
function PopoverPanel({ id, triggerRef, onClose, placement, align, offset, padding, children }) {
  const panelRef = useRef(null);
  const position = usePopoverPosition(triggerRef, panelRef, { placement, align, offset, padding });

  useEffect(() => {
    panelRef.current.focus({ preventScroll: true });
  }, []);

  useEffect(() => {
    const onPointerDown = (e) => {
      // the trigger toggles itself, so ignore it here or it would close and reopen
      if (!panelRef.current.contains(e.target) && !triggerRef.current.contains(e.target))
onClose(false);
    };
    const onKeyDown = (e) => {
      if (e.key === 'Escape') onClose(true);
    };
    document.addEventListener('pointerdown', onPointerDown);
    document.addEventListener('keydown', onKeyDown);
    return () => {
      document.removeEventListener('pointerdown', onPointerDown);
      document.removeEventListener('keydown', onKeyDown);
    };
  }, [onClose, triggerRef]);

  return (
    <div
      ref={panelRef}
      id={id}
      role="dialog"
      tabIndex={-1}
      data-placement={position?.placement}
      style={{
        position: 'fixed',
        top: position?.top ?? 0,
        left: position?.left ?? 0,
        maxHeight: position?.maxHeight,
        visibility: !position || position.hidden ? 'hidden' : 'visible',
      }}
      className="z-50 w-64 overflow-auto rounded-lg border border-slate-200 bg-white p-3 text-sm
shadow-xl outline-none"
    >
      {children}
    </div>
  );
}
```

```
export default function Popover({ label, children, placement = 'bottom', align = 'center', offset = 8, padding = 8 }) {
  const [open, setOpen] = useState(false);
  const triggerRef = useRef(null);
  const id = useId();

  const close = useCallback((returnFocus) => {
    setOpen(false);
    if (returnFocus) triggerRef.current.focus();
  }, []);

  return (
    <>
      <button
        ref={triggerRef}
        type="button"
        aria-haspopup="dialog"
        aria-expanded={open}
        aria-controls={open ? id : undefined}
        onClick={() => setOpen((o) => !o)}
        className="rounded-md bg-indigo-600 px-3 py-2 text-sm font-medium text-white hover:bg-indigo-500 focus-visible:outline-2 focus-visible:outline-offset-2 focus-visible:outline-indigo-600"
      >
        {label}
      </button>
      {open &&
        createPortal(
          <PopoverPanel
            id={id}
            triggerRef={triggerRef}
            onClose={close}
            placement={placement}
            align={align}
            offset={offset}
            padding={padding}
          >
            {children}
          </PopoverPanel>,
          document.body // escapes overflow: hidden and stacking contexts of ancestors
        )}
    </>
  );
}
```

`src/App.jsx`

```
import { useState } from 'react';
import Popover from './components/Popover';

const PLACEMENTS = ['top', 'bottom', 'left', 'right'];
const ALIGNS = ['start', 'center', 'end'];
const CORNERS = ['top-4 left-4', 'top-4 right-4', 'bottom-4 left-4', 'bottom-4 right-4'];
const MENU = ['Profile', 'Settings', 'Billing', 'Sign out'];

function Content() {
  return (
    <>
      <p className="font-semibold text-slate-900">Account</p>
      <p className="mt-1 text-slate-600">Flips when there is no room and slides to stay on screen.</p>
      <ul className="mt-2 space-y-1">
        {MENU.map((item) => (
          <li key={item}>
            <button type="button" className="w-full rounded px-2 py-1 text-left hover:bg-slate-100">
              {item}
            </button>
          </li>
        ))}
      </ul>
    </>
  );
}

export default function App() {
  const [placement, setPlacement] = useState('bottom');
  const [align, setAlign] = useState('center');
  const options = { placement, align };

  return (
    <main className="min-h-[200vh] bg-slate-50 text-slate-800">
      <section className="relative h-screen">
        {CORNERS.map((corner) => (
          <div key={corner} className={`absolute ${corner}`}>
            <Popover label="Open" {...options}>
              <Content />
            </Popover>
          </div>
        ))}

        <div className="absolute inset-0 m-auto flex h-fit w-fit flex-col items-center gap-3 rounded-xl
bg-white p-6 shadow">
          <h1 className="text-lg font-semibold">Popover positioning</h1>
```

```
          <label className="text-sm">
            Placement
            <select value={placement} onChange={(e) => setPlacement(e.target.value)} className="ml-2
rounded border px-2 py-1">
              {PLACEMENTS.map((p) => (
                <option key={p}>{p}</option>
              ))}
            </select>
          </label>
          <label className="text-sm">
            Align
            <select value={align} onChange={(e) => setAlign(e.target.value)} className="ml-2 rounded
border px-2 py-1">
              {ALIGNS.map((a) => (
                <option key={a}>{a}</option>
              ))}
            </select>
          </label>
          <Popover label="Center" {...options}>
            <Content />
          </Popover>
        </div>
      </section>

      <section className="mx-auto max-w-md p-4">
        <p className="mb-2 text-sm text-slate-500">Trigger inside an overflow: auto box (scroll it)</p>
        <div className="h-48 overflow-auto rounded-lg border bg-white p-4">
          <div className="h-[600px] pt-40">
            <Popover label="Scroll me" {...options}>
              <Content />
            </Popover>
          </div>
        </div>
      </section>
    </main>
  );
}
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
```

```
  <StrictMode>
    <App />
  </StrictMode>
);
```

src/index.css

```
@import "tailwindcss";
```

src/utils/computePosition.test.js

```
// Run: node src/utils/computePosition.test.js
import assert from 'node:assert/strict';
import { computePosition } from './computePosition.js';

const rect = (top, left, width, height) => ({ top, left, width, height, bottom: top + height, right:
left + width });
const vp = { width: 1000, height: 800 };
const pop = { width: 200, height: 150 };

// Fits below → stays bottom, centered under the trigger.
let r = computePosition({ triggerRect: rect(100, 100, 100, 30), popoverSize: pop, viewport: vp });
assert.deepEqual([r.placement, r.top, r.left], ['bottom', 138, 50]);

// Near the bottom edge → flips to top.
r = computePosition({ triggerRect: rect(700, 400, 100, 30), popoverSize: pop, viewport: vp });
assert.deepEqual([r.placement, r.top], ['top', 542]);

// Near the right edge with placement right → flips to left.
r = computePosition({ triggerRect: rect(300, 900, 60, 30), popoverSize: pop, viewport: vp, placement:
'right' });
assert.deepEqual([r.placement, r.left], ['left', 692]);

// Near the left edge → shifts right instead of going off-screen.
r = computePosition({ triggerRect: rect(100, 0, 40, 30), popoverSize: pop, viewport: vp });
assert.equal(r.left, 8);

// Neither side fits → keep the bigger side (bottom) and cap the height.
r = computePosition({ triggerRect: rect(100, 400, 100, 30), popoverSize: { width: 200, height: 250 },
viewport: { width: 1000, height: 300 } });
assert.deepEqual([r.placement, r.maxHeight], ['bottom', 154]);

// Flipped to top but still too tall → pinned to padding, height capped to fit above the trigger.
r = computePosition({ triggerRect: rect(200, 400, 100, 30), popoverSize: { width: 200, height: 250 },
viewport: { width: 1000, height: 300 } });
```

```
assert.deepEqual([r.placement, r.top, r.maxHeight], ['top', 8, 184]);

// Trigger scrolled out of view → hidden.
assert.equal(computePosition({ triggerRect: rect(-50, 100, 100, 30), popoverSize: pop, viewport: vp
}).hidden, true);

console.log('computePosition: all tests passed');
```

---

12. Wrap-up

Edge cases handled

- Flip on all four sides, only when the other side is actually bigger.
- Shift on the cross axis with viewport padding.
- Height cap when nothing fits, without a measure/cap feedback loop.
- Scroll of any ancestor, window resize, content resize.
- Trigger scrolled out of view → hidden.
- Escaping `overflow: hidden` via portal.
- No first-frame flicker (`useLayoutEffect` + hidden until measured).
- Outside click, Escape, focus return, ARIA links.

Trade-offs to mention

Decision	Why	Cost
`position: fixed` + portal	No scroll math, never clipped	Breaks inside an ancestor with `transform` / `filter` (it becomes the containing block).
Re-measure on scroll	Always correct	Layout read per frame while scrolling; fine for one popover.
Pure `computePosition`	Unit testable, reusable	Slightly more code than inline math.
`hidden` uses the viewport only	Simple	Trigger clipped by a scroll container (but still in viewport) is not detected.
Width never capped	Simple; text reflow is ugly	Popover wider than the viewport is pinned left.
No arrow	Not required	Easy to add from the same numbers (see Q11).

What to add with more time

- **Tests:** React Testing Library for open/close/Escape/focus; Playwright for real layout at corners (jsdom has no layout).
- **Arrow** pointing at the trigger center.
- **Auto placement:** pick the side with the most space.
- **Focus trap** / roving focus for menus (`role="menu"` , arrow keys).

- **Hover mode** for tooltips with open/close delays.
- **Animations** keyed on `data-placement` (slide from the correct side).
- **Native alternative:** HTML `popover` attribute + CSS Anchor Positioning (`position-try-fallbacks: flip-block`) where supported.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Why not just use Floating UI or Popper?

In production, I would. They handle iframes, transforms, virtual elements and many edge cases. In an interview the goal is to show I understand the math (rects, flip, shift) that those libraries implement. My `computePosition` has the same shape as Floating UI's middleware: place → flip → shift → size.

Q2. What is the difference between a tooltip, a popover and a dropdown here?

Positioning is the same. The difference is behavior and ARIA: a tooltip opens on hover/focus, is not interactive, and uses `role="tooltip"` + `aria-describedby`; a dropdown menu uses `role="menu"` with arrow-key navigation; a popover is a non-modal `role="dialog"` with free content.

Q3. Should the popover close on scroll instead of following?

It is a product choice. Following feels better for menus in long pages. Closing on scroll is simpler and avoids per-frame layout reads; native `<select>` dropdowns often do that. I'd ask the interviewer and keep the hook so both are one line apart.

State design

Q4. Why is the position in state and not just computed during render?

Render must be pure, and reading `getBoundingClientRect` in render reads the *previous* layout and can be wrong. Measuring happens after the DOM exists (layout effect), and the result must trigger one more render, so it needs state.

Q5. Why do you mount the panel only when open instead of toggling `display: none`?

A `display: none` element has no size, so it cannot be measured anyway. Mounting only when open also resets position to `null` each time and removes all scroll listeners when closed, so closed popovers cost nothing.

Q6. Why use refs for the DOM nodes and not state?

Refs don't cause re-renders and are always the latest node. The node never changes while mounted, so there is nothing to render on.

Challenge 1: flip and shift

Q7. What if neither side has enough space?

We stay on the side with more room and set `maxHeight` to that space minus offset and padding, so the content scrolls. For `top` we also pin `top` to `padding` so it doesn't go above the screen.

Q8. Why flip only when the opposite side is bigger?

Otherwise a trigger in the middle of a short window flips to a side that is even smaller, and while scrolling it can flip back and forth every frame (jitter). Comparing both sides makes the decision stable.

Q9. How would you add `auto` placement?

Compute the space on all four sides, subtract the needed size, and pick the side with the largest remaining space (prefer the requested one on ties). Then run the same shift and size steps.

Q10. Why `clamp` instead of flipping alignment from `start` to `end`?

Clamp handles every case with one line, including centered popovers. Switching alignment is nicer for menus aligned to a button edge; it's an extra step I can add before the clamp.

Q11. How would you add an arrow?

After shift, the arrow should still point at the trigger center: `arrowLeft = clamp(t.left + t.width / 2 - left, 12, p.width - 12)` for top/bottom (and the same with `top` / `height` for left/right). Render it outside the scrolling element so `overflow: auto` doesn't clip it, and rotate it based on `placement`.

Challenge 2: measuring

Q12. Why `useLayoutEffect` and not `useEffect`?

`useEffect` runs after paint, so the user sees one frame at the wrong position. `useLayoutEffect` runs after the DOM update but before paint; the state update inside it is flushed synchronously, so the first painted frame is already correct.

Q13. What about server-side rendering?

`useLayoutEffect` warns on the server and `document` doesn't exist. Here the panel only renders after a click, so it never renders on the server. For a popover open on first load, I'd render it after mount (`useEffect` sets a `mounted` flag).

Q14. Why `scrollHeight` for the height and not `getBoundingClientRect`?

Once we set `maxHeight`, the rect returns the capped height. Next measure it "fits", the cap is removed, and it oscillates. `scrollHeight` is the natural content height regardless of `maxHeight`, so the input to the math doesn't depend on its own output.

Challenge 3: scroll / resize

Q15. Scroll events don't bubble. How does a window listener see a scroll box scrolling?

They don't bubble, but they do go through the **capture** phase from `window` down to the target. `addEventListener('scroll', fn, true)` catches scrolls of every element.

Q16. Why `requestAnimationFrame` and not a debounce?

Debounce waits until scrolling stops, so the popover lags behind and then jumps. rAF runs at most once per frame, so it follows the trigger smoothly and never more often than the screen can show.

Q17. What if there are 50 popovers on the page?

Only open panels attach listeners, and normally one is open. If many must be open (e.g. tooltips in a table), share one scroll/resize listener and one rAF loop for all of them, and skip updates whose result didn't change.

Q18. What if the trigger moves without any scroll or resize (e.g. an animation or a layout shift)?

None of our listeners fire. Options: observe the trigger's parent with `ResizeObserver` / `MutationObserver`, or run a rAF loop while open (Floating UI's `animationFrame` option). The loop is the most correct and the most expensive.

Challenge 4: portal / fixed

Q19. When does `position: fixed` break?

If any ancestor of the fixed element has `transform`, `filter`, `perspective` or `contain: paint`, that ancestor becomes the containing block and `fixed` behaves like `absolute`. Portaling to `body` avoids that because the panel is no longer inside those ancestors.

Q20. Does the portal break event bubbling or context?

React events bubble through the React tree, not the DOM tree, so a click inside the portal still reaches the `Popover`'s React parents, and context works. DOM listeners on ancestors do not see it, which is why the outside-click check uses `panelRef.contains` and `triggerRef.contains`.

Q21. What about mobile with pinch zoom or the on-screen keyboard?

The layout viewport and visual viewport differ. Use `window.visualViewport` (width, height, offsetTop/Left) for the viewport and listen to its `resize` and `scroll` events.

Accessibility

Q22. How does a keyboard user use it?

Tab to the trigger, Enter/Space opens (it's a real `<button>`), focus moves into the panel, Tab reaches its buttons, Escape closes and returns focus to the trigger. `aria-expanded` tells screen readers the state, `aria-controls` links to the panel.

Q23. Why `pointerdown` for outside click and not `click`?

`pointerdown` fires before focus changes and before any `click` that might open something else, and it works for mouse, touch and pen. With `click`, a drag that starts inside and ends outside would close the popover.

Performance / trade-offs

Q24. Isn't calling `getBoundingClientRect` on every frame expensive?

It forces layout only if the layout is dirty. We read all rects first and then write styles once through React, so there is no read/write interleaving (no layout thrashing). One read per frame for one popover is cheap.

Q25. How would you test this?

The math is a pure function, so unit test it with plain numbers (done with `node:assert`). jsdom returns zeros for rects, so for components test behavior only (open, Escape, focus) with React Testing Library, and test real layout at screen corners with Playwright.

Q26. What is the modern native way to do this?

The HTML `popover` attribute gives top-layer rendering, light dismiss and Escape for free. CSS Anchor Positioning (`anchor-name`, `position-anchor`, `position-area`, `position-try-fallbacks: flip-block, flip-inline`) does placement and flip in CSS. Browser support decides whether we can drop the JS.